

Experiment 4

Aim :

To write and execute SQL queries using the Employee schema with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, simulated INTERSECT, and EXCEPT, and to compare execution plans using EXPLAIN.

Description :

This experiment demonstrates different SQL techniques for retrieving and comparing data from the Employee schema. JOIN operations are used to combine related records from multiple tables. An INNER JOIN returns only matching records, while a LEFT JOIN returns all records from the left table along with matching records from the right table. A self-join is used when a table is joined with itself, such as when employees are related to their managers.

A 3-way join combines three related tables in a single query. Correlated subqueries execute with reference to values from the outer query. EXISTS checks whether a related record exists, while simulated INTERSECT and EXCEPT operations can be achieved using EXISTS or NOT EXISTS when direct set operators are unavailable. EXPLAIN is used to view the query execution plan and understand how the database engine processes a query.

Source Code :

```
-- STEP 1: CREATE DATABASE
CREATE DATABASE IF NOT EXISTS DBMS_LAB;
USE DBMS_LAB;
```

```
-- STEP 2: REMOVE OLD TABLES IF THEY EXIST
DROP TABLE IF EXISTS Project;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Department;
```

```
-- STEP 3: CREATE DEPARTMENT TABLE
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50)
);
```

```
-- STEP 4: CREATE EMPLOYEE TABLE
CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    dept_id INT,
    manager_id INT,
    salary DECIMAL(10,2)
);
```

```
-- STEP 5: CREATE PROJECT TABLE
CREATE TABLE Project (
    project_id INT PRIMARY KEY,
```

```

    project_name VARCHAR(100),
    dept_id INT
);

-- STEP 6: INSERT DATA INTO DEPARTMENT
INSERT INTO Department (dept_id, dept_name)
VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance'),
(4, 'Marketing');

-- STEP 7: INSERT DATA INTO EMPLOYEE

INSERT INTO Employee
(emp_id, emp_name, dept_id, manager_id, salary)
VALUES
(101, 'Raj', 1, NULL, 80000),
(102, 'Aman', 1, 101, 60000),
(103, 'Priya', 2, NULL, 70000),
(104, 'Rohan', 2, 103, 50000),
(105, 'Neha', 3, NULL, 75000),
(106, 'Karan', 3, 105, 55000),
(107, 'Simran', 4, NULL, 65000),
(108, 'Arjun', 1, 101, 55000);

-- STEP 8: INSERT DATA INTO PROJECT
INSERT INTO Project
(project_id, project_name, dept_id)
VALUES
(201, 'Website Development', 1),
(202, 'Mobile Application', 1),
(203, 'Recruitment System', 2),
(204, 'Financial Analysis', 3),
(205, 'Marketing Campaign', 4);

-- STEP 9: DISPLAY TABLE DATA
SELECT * FROM Department;

SELECT * FROM Employee;

SELECT * FROM Project;

```

byteXL MySQL Run AI Agent 79 ms

```

63 -- STEP 7: INSERT DATA INTO EMPLOYEE
64
65
66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- STEP 8: INSERT DATA INTO PROJECT
80
81
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94 -- STEP 9: DISPLAY TABLE DATA
95
96
97 SELECT * FROM Department;
98
99 SELECT * FROM Employee;
100
101 SELECT * FROM Project;
102
103
104

```

STON

Input for the program (Optional)

Output:

dept_id	dept_name
1	IT
2	HR
3	Finance
4	Marketing

emp_id	emp_name	dept_id	manager_id	salary
101	Raj	1	NULL	80000.00
102	Aman	1	101	60000.00
103	Priya	2	NULL	70000.00
104	Rohan	2	103	50000.00
105	Neha	3	NULL	75000.00
106	Karan	3	105	55000.00
107	Simran	4	NULL	65000.00
108	Arjun	1	101	55000.00

project_id	project_name	dept_id
201	Website Development	1
202	Mobile Application	1
203	Recruitment System	2
204	Financial Analysis	3
205	Marketing Campaign	4

Ready Feedback New History Save

-- 1. INNER JOIN

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;

byteXL MySQL Run AI Agent 74 ms

```

62
63 -- STEP 7: INSERT DATA INTO EMPLOYEE
64
65
66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- STEP 8: INSERT DATA INTO PROJECT
80
81
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94
95 -- 1. INNER JOIN
96
97
98 SELECT e.emp_id, e.emp_name, d.dept_name
99 FROM Employee e
100 INNER JOIN Department d
101 ON e.dept_id = d.dept_id;
102
103

```

STON

Input for the program (Optional)

Output:

emp_id	emp_name	dept_name
101	Raj	IT
102	Aman	IT
103	Priya	HR
104	Rohan	HR
105	Neha	Finance
106	Karan	Finance
107	Simran	Marketing
108	Arjun	IT

Ready Feedback New History Save

-- 2. LEFT JOIN

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

LEFT JOIN Department d

ON e.dept_id = d.dept_id;

The screenshot shows the byteXL MySQL IDE interface. The query window contains the following SQL code:

```

62 -- STEP 7: INSERT DATA INTO EMPLOYEE
63 --
64 --
65
66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- STEP 8: INSERT DATA INTO PROJECT
80 --
81 --
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94 -- 2. LEFT JOIN
95 --
96 --
97
98 SELECT e.emp_id, e.emp_name, d.dept_name
99 FROM Employee e
100 LEFT JOIN Department d
101 ON e.dept_id = d.dept_id;
102
103
104

```

The results window shows the output of the query, displaying a table with columns: emp_id, emp_name, dept_name. The output is as follows:

emp_id	emp_name	dept_name
101	Raj	IT
102	Aman	IT
103	Priya	HR
104	Rohan	HR
105	Neha	Finance
106	Karan	Finance
107	Simran	Marketing
108	Arjun	IT

-- 3. SELF JOIN: EMPLOYEE AND MANAGER

```

SELECT e.emp_name AS Employee,
       m.emp_name AS Manager
FROM Employee e
LEFT JOIN Employee m
ON e.manager_id = m.emp_id;

```

The screenshot shows the byteXL MySQL IDE interface. The query window contains the following SQL code:

```

63 -- STEP 7: INSERT DATA INTO EMPLOYEE
64 --
65 --
66 --
67 --
68 --
69 --
70 --
71 --
72 --
73 --
74 --
75 --
76 --
77 --
78 --
79 -- STEP 8: INSERT DATA INTO PROJECT
80 --
81 --
82 --
83 --
84 --
85 --
86 --
87 --
88 --
89 --
90 --
91 --
92 --
93 --
94 -- 3. SELF JOIN: EMPLOYEE AND MANAGER
95 --
96 --
97 --
98 SELECT e.emp_name AS Employee,
99        m.emp_name AS Manager
100 FROM Employee e
101 LEFT JOIN Employee m
102 ON e.manager_id = m.emp_id;
103
104

```

The results window shows the output of the query, displaying a table with columns: Employee, Manager. The output is as follows:

Employee	Manager
Raj	NULL
Aman	Raj
Priya	NULL
Rohan	Priya
Neha	NULL
Karan	Neha
Simran	NULL
Arjun	Raj

-- 4. THREE-WAY JOIN

```

SELECT e.emp_name,
       d.dept_name,
       p.project_name
FROM Employee e

```

INNER JOIN Department d
 ON e.dept_id = d.dept_id
 INNER JOIN Project p
 ON d.dept_id = p.dept_id;

The screenshot shows the byteXL SQL editor interface. The left pane contains a MySQL query with line numbers 66 to 107. The query includes two INSERT statements and a SELECT statement with three INNER JOINs. The right pane shows the output of the query, which is a table with three columns: emp_name, dept_name, and project_name. The output table contains 10 rows of data.

```

66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 101, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 --
80 -- STEP B: INSERT DATA INTO PROJECT
81 --
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94 --
95 -- 4. THREE-WAY JOIN
96 --
97
98 SELECT e.emp_name,
99        d.dept_name,
100        p.project_name
101 FROM Employee e
102 INNER JOIN Department d
103 ON e.dept_id = d.dept_id
104 INNER JOIN Project p
105 ON d.dept_id = p.dept_id;
106
107
  
```

Output:

emp_name	dept_name	project_name
Raj	IT	Mobile Application
Raj	IT	Website Development
Aman	IT	Mobile Application
Aman	IT	Website Development
Priya	HR	Recruitment System
Rohan	HR	Recruitment System
Neha	Finance	Financial Analysis
Karan	Finance	Financial Analysis
Simran	Marketing	Marketing Campaign
Arjun	IT	Mobile Application
Arjun	IT	Website Development

-- 5. CORRELATED SUBQUERY
 SELECT e.emp_id,
 e.emp_name,
 e.salary
 FROM Employee e
 WHERE e.salary > (
 SELECT AVG(e2.salary)
 FROM Employee e2
 WHERE e2.dept_id = e.dept_id
);

byteXL

queries.sql x + MySQL Run 81 ms

```

67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79
80 -- STEP 8: INSERT DATA INTO PROJECT
81
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94
95 -- 5. CORRELATED SUBQUERY
96
97
98 SELECT e.emp_id,
99        e.emp_name,
100        e.salary
101 FROM Employee e
102 WHERE e.salary > (
103     SELECT AVG(e2.salary)
104     FROM Employee e2
105     WHERE e2.dept_id = e.dept_id
106 );
107
108

```

STON

Input for the program (Optional)

Output

emp_id	emp_name	salary
101	Raj	80000.00
103	Priya	70000.00
105	Neha	75000.00

Ready

Feedback

New History Save

```

-- 6. EXISTS
SELECT e.emp_id,
       e.emp_name
FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Project p
    WHERE p.dept_id = e.dept_id
);

```

byteXL

queries.sql x + MySQL Run 65 ms

```

66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79
80 -- STEP 8: INSERT DATA INTO PROJECT
81
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94
95 -- 6. EXISTS
96
97
98 SELECT e.emp_id,
99        e.emp_name
100 FROM Employee e
101 WHERE EXISTS (
102     SELECT 1
103     FROM Project p
104     WHERE p.dept_id = e.dept_id
105 );
106
107

```

STON

Input for the program (Optional)

Output

emp_id	emp_name
101	Raj
102	Aman
103	Priya
104	Rohan
105	Neha
106	Karan
107	Simran
108	Arjun

Ready

Feedback

New History Save

```

-- 7. SIMULATED INTERSECT
SELECT e.emp_id,

```

```

        e.emp_name
FROM Employee e
WHERE EXISTS (
    SELECT 1
    FROM Project p
    WHERE p.dept_id = e.dept_id
);

```

The screenshot shows the ByteXL SQL editor interface. The left pane displays a MySQL query with the following content:

```

66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- =====
80 -- STEP 8: INSERT DATA INTO PROJECT
81 -- =====
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93 -- =====
94 -- 7. SIMULATED INTERSECT
95 -- =====
96
97 SELECT e.emp_id,
98        e.emp_name
99 FROM Employee e
100 WHERE EXISTS (
101     SELECT 1
102     FROM Project p
103     WHERE p.dept_id = e.dept_id
104 );
105
106
107

```

The right pane shows the output of the query, which is a table with two columns: emp_id and emp_name. The output is as follows:

emp_id	emp_name
101	Raj
102	Aman
103	Priya
104	Rohan
105	Neha
106	Karan
107	Simran
108	Arjun

The interface also includes a top bar with the ByteXL logo, a search bar, and a user profile icon labeled YUVRAJ. The bottom bar has buttons for Feedback, New, History, and Save.

```

-- 8. SIMULATED EXCEPT
SELECT e.emp_id,
       e.emp_name
FROM Employee e
WHERE NOT EXISTS (
    SELECT 1
    FROM Project p
    WHERE p.dept_id = e.dept_id
);

```

byteXL

queries.sql x + MySQL Run 91 ms

```

68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- -----
80 -- STEP 8: INSERT DATA INTO PROJECT
81 -- -----
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94
95 -- -----
96 -- 8. SIMULATED EXCEPT
97 -- -----
98
99 SELECT e.emp_id,
100        e.emp_name
101 FROM Employee e
102 WHERE NOT EXISTS (
103     SELECT 1
104     FROM Project p
105     WHERE p.dept_id = e.dept_id
106 );
107
108
109

```

STDOUT

Input for the program (Optional)

Output

Program did not output anything!

Ready

Feedback

New History Save

```

-- 9. EXPLAIN
EXPLAIN
SELECT e.emp_id,
       e.emp_name,
       d.dept_name
FROM Employee e
INNER JOIN Department d
ON e.dept_id = d.dept_id;

```

byteXL

queries.sql x + MySQL Run 91 ms

```

65
66 INSERT INTO Employee
67 (emp_id, emp_name, dept_id, manager_id, salary)
68 VALUES
69 (101, 'Raj', 1, NULL, 80000),
70 (102, 'Aman', 1, 101, 60000),
71 (103, 'Priya', 2, NULL, 70000),
72 (104, 'Rohan', 2, 103, 50000),
73 (105, 'Neha', 3, NULL, 75000),
74 (106, 'Karan', 3, 105, 55000),
75 (107, 'Simran', 4, NULL, 65000),
76 (108, 'Arjun', 1, 101, 55000);
77
78
79 -- -----
80 -- STEP 8: INSERT DATA INTO PROJECT
81 -- -----
82
83 INSERT INTO Project
84 (project_id, project_name, dept_id)
85 VALUES
86 (201, 'Website Development', 1),
87 (202, 'Mobile Application', 1),
88 (203, 'Recruitment System', 2),
89 (204, 'Financial Analysis', 3),
90 (205, 'Marketing Campaign', 4);
91
92
93
94
95 -- -----
96 -- 9. EXPLAIN
97 -- -----
98
99 EXPLAIN
100 SELECT e.emp_id,
101        e.emp_name,
102        d.dept_name
103 FROM Employee e
104 INNER JOIN Department d
105 ON e.dept_id = d.dept_id;
106

```

STDOUT

Input for the program (Optional)

Output

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	e	NONE	ALL	NONE	NONE	NONE	NONE	8	100.00	Using where
1	SIMPLE	d	NONE	eq_ref	PRIMARY	PRIMARY	4	sandbox_db.e.dept_id	1	100.00	NONE

Ready

Feedback

New History Save